Learning Which VRP Solver Works Best

Machine-learning algorithm selection for the Capacitated Vehicle Routing Problem (CVRP).

Given measurable features of an instance (size, demand variance, depot centrality, ...), a classifier predicts which classical heuristic will yield the best solution within a fixed 20-second time budget. The problem is introduced by solving benchmark and self-created instances both exactly (gurobipy) and heuristically. Methodology is inspired by and adapted from Asín-Achá et al. (2024), *"Selecting fast algorithms for the capacitated vehicle routing problem with machine learning techniques"*, Networks 84(4):465-480, at student scale.

This notebook is the compact, executable record of the whole project. It runs in **fast mode**: every cell loads the frozen result artifacts from `reports/` instead of re-running the expensive experiments (re-running is documented at the end). Full prose: `PROJECT_REPORT.md`.

1. Libraries and Environment

Library	Role
`verypy` (pinned Git commit `8698f910`)	Classical CVRP construction heuristics (the solver portfolio)
`gurobipy` ≥ 13 (free size-limited license)	Exact MIP baseline for the problem introduction
`pyvrp` ≥ 0.13	Hybrid genetic search — external speed-vs-quality reference
`numpy`, `pandas`	Numerics and tabular experiment data
`scikit-learn`	Classifiers (Random Forest, KNN, Decision Tree, Gradient Boosting), metrics, grid search
`matplotlib`	Frozen result plots
`pytest`	Test suite (`uv run pytest`)

Dependencies are managed with `uv` (`uv sync`); Python ≥ 3.11. The exact solver and PyVRP are only needed to re-run those experiments. Most notebook cells only load frozen pandas/numpy artifacts; the integrity-check cell also imports the local project package.

In [1]:
```
import json
import math
import platform
from pathlib import Path

import numpy as np
import pandas as pd

print("python", platform.python_version(), "| numpy", np.__version__, "| pandas"
BASE = Path("reports/final_ml_experiment")
```

Loading [MathJax]/extensions/Safe.js

```
assert BASE.exists(), "run from the repository root"
summary = json.loads((BASE / "ml_summary.json").read_text())
metrics = pd.read_csv(BASE / "model_metrics.csv")
experiment = pd.read_csv(BASE / "experiment_table.csv")
features = json.loads((BASE / "feature_columns.json").read_text())
exp_summary = json.loads((BASE / "summary.json").read_text())
print(f"frozen experiment: {len(experiment)} instances, {len(features)} features
```

```
python 3.12.13 | numpy 2.5.1 | pandas 3.0.3
frozen experiment: 1260 instances, 38 features
```

2. Scope and Assumptions

- **Problem class**: symmetric Euclidean CVRP. One depot, homogeneous vehicle capacity, heterogeneous positive customer demands, unlimited fleet. Routes start and end at the depot.
- **Distance convention**: `cvrplib_euc_2d` — TSPLIB-style rounded distances, `floor(d + 0.5)`. Exact solver, heuristics, validation, and features all use the instance's own convention, so costs are directly comparable (and comparable to CVRPLIB best-known integer values).
- **Fixed time budget**: every portfolio solver runs under the same 20-second wall clock. The label means "best solution within the budget"; a timeout is recorded as a failure. Exact-cost ties break toward the faster solver (as in the paper).
- **Labels are portfolio-relative**: the "best solver" is the best of the five portfolio heuristics, not a proven optimum. The Virtual Best Solver (VBS) is a portfolio oracle.
- **Training data is synthetic** by design (the paper's protocol): a balanced factorial grid meant to span instance variation. Real CVRPLIB benchmarks serve as the out-of-distribution test.
- **No leakage**: model features are pre-solution statistics only; solver outputs, labels, seeds, and generator category names are rejected from the feature matrix by explicit code checks.
- **Independent verification**: every solution (heuristic, exact, or genetic) is re-validated and re-costed by project-owned code, never trusted from the library.
- Deliberately out of scope: VRPTW, pickup & delivery, heterogeneous fleets, reinforcement learning, custom metaheuristic implementations.

3. Problem Introduction and Exact Baseline

The CVRP is NP-hard. `exact.py` formulates it as a MIP (gurobipy): a two-index arc formulation with binary arc variables x_{ij} and load variables $u_i \in [q_i, Q]$, where $u_j \ge u_i + q_j - Q(1 - x_{ij})$ propagates load along arcs — eliminating subtours and enforcing capacity simultaneously.

Three small instances (a 6-customer manual instance and two 20-customer synthetic instances from the final dataset design) are solved exactly (300 s limit) and with all five portfolio heuristics. The pattern below motivates the project: at n=6 the optimum is proven instantly and all heuristics match it; at n=20 the MIP cannot close a 0.7–2.1% gap in 300 s while heuristics match its incumbent in milliseconds; at n=51 the model exceeds the free license (2,550 binary variables > 2,000). Exact solving is the reference, not the workhorse.

Loading [MathJax]/extensions/Safe.js

In [2]:
```
exact = pd.read_csv("reports/exact_baseline/exact_vs_heuristic.csv")
exact[["instance", "n_customers", "method", "cost", "runtime_seconds", "optimal"
```

Out[2]:

	instance	n_customers	method	cost	runtime_seconds	optimal	gap_to_ex
0	tiny-manual	6	gurobipy-MTZ (exact)	50.828427	0.012000	True	
1	tiny-manual	6	CW64-PS/minimize-distance+2opt1	50.828427	0.000712	False	
2	tiny-manual	6	MJ76-insertion/all-strain	50.828427	0.002251	False	
3	tiny-manual	6	WH72-sweep/both-directions	50.828427	0.003283	False	
4	tiny-manual	6	CMT79-2phase/default	50.828427	0.000849	False	
5	tiny-manual	6	Be83-RFCS/default	50.828427	0.000634	False	
6	syn-n20-uniform-center-low_variance-s0-df2e475...	20	gurobipy-MTZ (exact)	524.000000	300.046000	False	
7	syn-n20-uniform-center-low_variance-s0-df2e475...	20	CW64-PS/minimize-distance+2opt1	539.000000	0.001212	False	
8	syn-n20-uniform-center-low_variance-s0-df2e475...	20	MJ76-insertion/all-strain	524.000000	0.015126	False	
9	syn-n20-uniform-center-low_variance-s0-df2e475...	20	WH72-sweep/both-directions	524.000000	0.018252	False	
10	syn-n20-uniform-center-low_variance-s0-df2e475...	20	CMT79-2phase/default	524.000000	0.002200	False	
11	syn-n20-uniform-center-low_variance-s0-df2e475...	20	Be83-RFCS/default	524.000000	0.010924	False	
12	syn-n20-corner-	20	gurobipy-MTZ (exact)	571.000000	300.013000	False	

Loading [MathJax]/extensions/Safe.js

	instance	n_customers	method	cost	runtime_seconds	optimal	gap_to_ex
	high_variance-s0-a...						
13	syn-n20-two_clusters-corner-high_variance-s0-a...	20	CW64-PS/minimize-distance+2opt1	575.000000	0.001409	False	
14	syn-n20-two_clusters-corner-high_variance-s0-a...	20	MJ76-insertion/all-strain	574.000000	0.014713	False	
15	syn-n20-two_clusters-corner-high_variance-s0-a...	20	WH72-sweep/both-directions	574.000000	0.039600	False	
16	syn-n20-two_clusters-corner-high_variance-s0-a...	20	CMT79-2phase/default	574.000000	0.002095	False	
17	syn-n20-two_clusters-corner-high_variance-s0-a...	20	Be83-RFCS/default	680.000000	0.007726	False	

Benchmark instance E-n51-k5

The CVRPLIB benchmark E-n51-k5 (proven optimum 521) is solved heuristically; the Random Forest selector — trained only on synthetic data — predicts which portfolio solver wins.

In [3]:
```
demo = json.loads(Path("reports/exact_baseline/benchmark_demo.json").read_text()
print("best known:", demo["best_known_cost"])
print("exact attempt:", demo["exact_attempt"]["failure_reason"])
print("selector predicted:", demo["selector"]["predicted_solver"],
      "| prediction is best in portfolio:", demo["selector"]["prediction_is_best
pd.DataFrame(demo["heuristics"])[["method", "cost", "gap_to_best_known_percent",
```

```
best known: 521.0
exact attempt: GurobiError: Model too large for size-limited license; visit http
s://gurobi.com/unrestricted for more information
selector predicted: WH72-sweep/both-directions | prediction is best in portfoli
o: True
```

Out[3]:

	method	cost	gap_to_best_known_percent	runtime_seconds
0	CW64-PS/minimize-distance+2opt1	580.0	11.324376	0.003948
1	MJ76-insertion/all-strain	555.0	6.525912	0.069411
2	WH72-sweep/both-directions	522.0	0.191939	0.164788
	)-2phase/default	627.0	20.345489	0.005133

Loading [MathJax]/extensions/Safe.js

	method	cost	gap_to_best_known_percent	runtime_seconds
4	Be83-RFCS/default	567.0	8.829175	0.124252

4. Data

Synthetic training/test data (`synthetic.py`): a balanced factorial grid — 5 spatial families × 3 depot positions × 4 demand regimes × 3 sizes (20/50/100 customers) × 7 seeds = **1,260 instances**, each regenerable from a stable ID; generation manifests are saved with the results. Labelling required 6,300 timed solver runs (exactly one Wren-Holliday timeout).

Real benchmark data (`data/cvrplib/`): E-n51-k5 plus the classic CVRPLIB sets A, B, E, P (87 files with `.sol` optima; 85 usable — two E instances lack Euclidean coordinates).

In [4]:
```
design = experiment[["spatial_family", "depot_position", "demand_regime", "custo
print(design.to_string())
print("\ndistance convention:", experiment["distance_convention"].unique())
print("\nlabel distribution (best solver within 20 s):")
print(experiment["best_solver"].value_counts().to_string())
print("\nsolver failures:", summary["solver_failure_counts"])
```

```
spatial_family    5
depot_position    3
demand_regime     4
customer_count    3
seed              7

distance convention: <StringArray>
['cvrplib_euc_2d']
Length: 1, dtype: str

label distribution (best solver within 20 s):
best_solver
Be83-RFCS/default                  323
CW64-PS/minimize-distance+2opt1    314
MJ76-insertion/all-strain          266
WH72-sweep/both-directions         238
CMT79-2phase/default               119

solver failures: {'Be83-RFCS/default': 0, 'CMT79-2phase/default': 0, 'CW64-PS/mi
nimize-distance+2opt1': 0, 'MJ76-insertion/all-strain': 0, 'WH72-sweep/both-dire
ctions': 1}
```

5. Solver Portfolio

Five deterministic VeRyPy heuristic configurations, spanning the classic construction families:

Configuration	Family (year)
`CW64-PS/minimize-distance+2opt1`	Clarke–Wright parallel savings + intra-route 2-opt (1964)
`WH72-sweep/both-directions`	Wren–Holliday sweep (1972)
`/all-strain`	Mole–Jameson insertion (1976)

Loading [MathJax]/extensions/Safe.js

Configuration	Family (year)
`CMT79-2phase/default`	Christofides–Mingozzi–Toth two-phase (1979)
`Be83-RFCS/default`	Beasley route-first cluster-second (1983)

Two deliberate exclusions: **Gaskell savings** (won only 3/240 audit instances — not a learnable class; see `reports/portfolio_audit/`) and **hybrid genetic search** (would win nearly every label and collapse the class distribution — the pipeline enforces an 80% dominance ceiling; HGS is instead benchmarked as an external reference in Section 11). Treating configurations as separate portfolio entries is the paper's discretized form of algorithm configuration.

6. Features (38, pre-solution only)

Engineered around three hypotheses about what drives solver performance: **(1) demand structure** — capacity pressure, demand mean/std/CV/quantiles; **(2) depot geometry** — normalized depot centrality, depot-distance statistics, and the paper's "circular convolution": customer share in 5 concentric rings around the depot; **(3) customer layout & size** — n, bounding box, pairwise/nearest-neighbor spread, cluster separation. A blocklist in `ml_experiment.py` rejects any solver-outcome or generator column.

In [5]:
```
print(len(features), "features; first 10:", features[:10])
prohibited = ["cost", "runtime", "feasible", "failure", "best_solver", "instance
              "spatial_family", "depot_position", "demand_regime"]
assert not [c for c in features if any(p in c for p in prohibited)], "leakage!"
print("leakage check passed: no solver/outcome/generator columns in the feature
```

```
38 features; first 10: ['bounding_box_area', 'capacity', 'capacity_pressure', 'c
luster_separation_estimate', 'coordinate_height', 'coordinate_width', 'customer_
density', 'demand_cv', 'demand_max', 'demand_mean']
leakage check passed: no solver/outcome/generator columns in the feature matrix
```

7. Evaluation Setup

- **SBS** (single best solver): best average solver on the *training* split — the no-ML baseline.
- **VBS** (virtual best solver): per-instance oracle — the ceiling.
- **Regret**: (selected cost − VBS cost) / VBS cost. Also reported: share within 1% / 5% of VBS.
- **m̂** (paper's normalized metric): (m_s − m_VBS)/(m_SBS − m_VBS) over mean cost ratios — 0 = oracle-quality selection, 1 = no better than SBS, > 1 = worse than no ML.
- Split: stratified 80/20 (1,008 train / 252 test, disjoint IDs). Four model families + two dummy baselines; fixed hyperparameters (validated post-hoc by grid search, Section 10).
- Deployment fallback: if the predicted solver fails on an instance, the meta-solver runs the SBS.

In [6]:
```
sbs_ratio = float(metrics.loc[metrics["method"] == "SBS", "mean_cost_ratio"].ilo
metrics["m_hat"] = ((metrics["mean_cost_ratio"] - 1.0) / (sbs_ratio - 1.0)).roun
"method", "accuracy", "balanced_accuracy", "mean_regret", "mean_
```

Loading [MathJax]/extensions/Safe.js

```
                "within_1_percent", "within_5_percent", "m_hat"]].sort_values("m
view.round(4)
```

Out[6]:

	method	accuracy	balanced_accuracy	mean_regret	mean_cost_ratio	within_1_percen
0	VBS	1.0000	1.0000	0.0000	1.0000	1.000
1	RandomForest	0.4008	0.3785	0.0124	1.0124	0.631
2	KNN	0.3849	0.3588	0.0132	1.0132	0.607
3	GradientBoosting	0.3452	0.3058	0.0137	1.0137	0.587
4	SBS	0.2103	0.2000	0.0167	1.0167	0.452
5	DecisionTree	0.3214	0.2876	0.0175	1.0175	0.551
6	Dummy-most-frequent	0.2579	0.2000	0.0216	1.0216	0.500
7	Dummy-stratified	0.2341	0.2043	0.0264	1.0264	0.448

8. Main Result

Random Forest is the best selector (as in the paper): mean regret 1.24% vs SBS 1.67%, within-1%-of-VBS 63.1% vs 45.2%, $\hat{m} = 0.741$ — it closes ≈26% of the baseline→oracle gap. Classification accuracy is modest (40.1% over 5 classes) because near-ties between solvers make the label cheap to miss — routing regret is the metric that matters. The plain Decision Tree ($\hat{m} > 1$) shows a weak selector is *worse* than no selector.

 within thresholds

9. Feature Importance: Expected vs Observed *(the core finding)*

The two feature-importance views tell a consistent but non-causal story:

1. **Demand structure is prominent in the Random Forest impurity ranking**, especially capacity pressure and demand spread.
2. **Held-out permutation importance highlights depot geometry and radial ring occupancy** as the most robust predictors.
3. **Raw size is surprisingly weak**: it never appears in the top 10. All five heuristics handle n ≤ 100 easily; the *shape* of the instance separates them more than its scale.

The observed win-rate breakdowns are broadly consistent with the heuristics' construction principles, but they should not be read as causal proof.

In [7]:

```
imp = pd.read_csv(BASE / "random_forest_feature_importance.csv").head(5)
perm = pd.read_csv(BASE / "permutation_importance.csv").head(5)
print("top-5 impurity importance:      ", ", ".join(imp["feature"]))
print("top-5 permutation (held-out):   ", ", ".join(perm["feature"]))
```

Loading [MathJax]/extensions/Safe.js

```
def top_shares(field):
    rows = {}
    for value, counts in exp_summary["label_breakdowns"][field].items():
        total = sum(counts.values())
        best = max(counts.items(), key=lambda item: item[1])
        rows[value] = f"{best[0].split('/')[0]} ({best[1] / total:.0%})"
    return rows

print("\ndominant solver by demand regime:", top_shares("demand_regime"))
print("dominant solver by depot position:", top_shares("depot_position"))
```

```
top-5 impurity importance:       capacity_pressure, demand_std, depot_distance_s
td, depot_centrality_normalized, total_demand
top-5 permutation (held-out):    depot_centrality_normalized, ring_3_ratio, depo
t_distance_std, ring_4_ratio, std_pairwise_customer_distance

dominant solver by demand regime: {'high_pressure': 'MJ76-insertion (34%)', 'hig
h_variance': 'CW64-PS (38%)', 'low_pressure': 'Be83-RFCS (37%)', 'low_variance':
'Be83-RFCS (41%)'}
dominant solver by depot position: {'center': 'Be83-RFCS (26%)', 'corner': 'MJ76
-insertion (28%)', 'random': 'CW64-PS (27%)'}
```

Reading: even demands → route-first-cluster-second (41% — splitting a giant tour is clean); lumpy demands → Clarke-Wright+2-opt (38% — savings merges absorb variance); tight capacity → Mole-Jameson (34% — feasibility-aware insertion); central depots/radial layouts → sweep peaks (needs angular spread), while sweep becomes less competitive for corner-depot instances.

 feature importance

Grouped selector performance

Strongest on 100-customer instances (more structure to read) and weakest under high capacity pressure. Random depot placement has the lowest accuracy, although regret remains similar to other depot groups.

In [8]:

```
grouped = json.loads((BASE / "grouped_metrics.json").read_text())
pd.concat({field: pd.DataFrame(grouped[field]).T[["n", "accuracy", "mean_regret"
           for field in ("customer_count", "demand_regime", "depot_position")}).
```

Out[8]:

		n	accuracy	mean_regret	within_1_percent
customer_count	**100**	83.0	0.482	0.007	0.711
	20	81.0	0.370	0.017	0.580
	50	88.0	0.352	0.013	0.602
demand_regime	**high_pressure**	58.0	0.293	0.021	0.466
	high_variance	73.0	0.452	0.011	0.685
	low_pressure	64.0	0.453	0.009	0.688
	low_variance	57.0	0.386	0.010	0.667
depot_position	**center**	92.0	0.424	0.014	0.609

Loading [MathJax]/extensions/Safe.js

	n	accuracy	mean_regret	within_1_percent
corner	76.0	0.447	0.011	0.697
random	84.0	0.333	0.011	0.595

10. Robustness Checks

Out-of-distribution (CVRPLIB A/B/E/P, 85 evaluated real instances) — the paper's protocol of testing on public benchmarks. In total, 87 candidates were collected, of which 85 were usable because two E instances lack Euclidean coordinates. Transfer is *partial and honestly reported*: within-1% rate still beats SBS (52.9% vs 43.5%, better on every set) but mean regret is essentially tied ($\hat{m} = 0.98$; best set B at 0.85). Same phenomenon the paper reports for stylized benchmark families. The portfolio VBS averages +1.98% vs proven optima; E-n51-k5 is an illustrative single benchmark case where the selector picks the portfolio winner.

Hyperparameter grid search (paper-style, 5-fold stratified CV on the frozen training split): all CV accuracies land in 0.392-0.404 and tuned test regrets straddle the frozen Random Forest's 1.24% — the fixed hyperparameters were sound; frozen results stand.

In [9]:
```
ood = json.loads(Path("reports/cvrplib_ood/cvrplib_ood_summary.json").read_text(
print("OOD overall:", {k: round(v, 3) for k, v in ood["overall"].items() if isin
print("OOD m-hat per set:", {k: round(v["m_hat"], 2) for k, v in ood["per_set"].
tuning = json.loads(Path("reports/hyperparameter_tuning/tuning_summary.json").re
print("grid-search CV accuracy:", {k: round(v["cv_accuracy"], 3) for k, v in tun
```

```
OOD overall: {'m_hat': 0.981, 'rf_accuracy': 0.318, 'rf_mean_cost_ratio': 1.018,
'rf_mean_gap_to_optimal_percent': 3.853, 'rf_mean_regret': 0.018, 'rf_within_1_p
ercent': 0.529, 'sbs_mean_cost_ratio': 1.019, 'sbs_mean_regret': 0.019, 'sbs_wit
hin_1_percent': 0.435, 'vbs_mean_gap_to_optimal_percent': 1.978}
OOD m-hat per set: {'A': 1.0, 'B': 0.85, 'E': 1.1, 'P': 1.0}
grid-search CV accuracy: {'DecisionTree': 0.402, 'GradientBoosting': 0.394, 'KN
N': 0.404, 'RandomForest': 0.392}
```

11. Extension: Speed vs Quality (PyVRP HGS at the same budget)

PyVRP's hybrid genetic search is run with the full 20 s on a stratified 45-instance subsample of the test split plus E-n51-k5. Here, VBS is only the oracle over the five classical portfolio solvers; HGS is an external high-quality reference, not part of the main portfolio. HGS wins/ties the portfolio oracle on **100%** of sampled instances and produces routes about **3.5% shorter** than the ML-selected heuristic and about **2.1% shorter** than the five-solver portfolio VBS; it also finds the proven optimum on E-n51-k5. The cost is compute: HGS consumes the entire 20 s budget, versus 0.22 s for the ML-selected heuristic (**≈91×** compute). This quantifies why HGS stays out of the selector portfolio and what regime each tool serves: real-time answers / warm starts → selector; 20 s per instance affordable → HGS.

```
(Path("reports/hgs_benchmark/hgs_summary.json").read_text())
print({k: round(v, 4) if isinstance(v, float) else v
```

Loading [MathJax]/extensions/Safe.js

```
        for k, v in hgs.items() if k not in ("environment", "sampling")})
print(json.loads(Path("reports/hgs_benchmark/hgs_e_n51_k5.json").read_text()))
```

```
{'hgs_seed': 0, 'hgs_win_or_tie_rate_vs_vbs': 1.0, 'mean_hgs_gain_vs_rf_selecte
d': 0.0349, 'mean_hgs_gain_vs_vbs': 0.0215, 'mean_hgs_runtime': 20.003, 'mean_rf
_selected_runtime': 0.2191, 'median_hgs_gain_vs_rf_selected': 0.0353, 'n_hgs_fea
sible': 45, 'n_instances': 45, 'runtime_ratio_hgs_over_rf_selected': 91.2999, 's
bs_solver': 'MJ76-insertion/all-strain', 'time_budget_seconds': 20.0}
{'benchmark': 'E-n51-k5', 'best_known_cost': 521.0, 'hgs': {'cost': 521.0, 'feas
ible': True, 'n_routes': 5, 'runtime_seconds': 20.0024427999997}, 'hgs_gap_to_be
st_known_percent': 0.0, 'time_budget_seconds': 20.0}
```

12. Integrity Checks

Re-derives every frozen label from raw costs, and asserts the oracle invariants.

In [11]:

```
from cvrp_algorithm_selection.ml_experiment import final_solver_prefixes, valida

validate_label_minima(experiment, final_solver_prefixes())
vbs = metrics[metrics["method"] == "VBS"].iloc[0]
assert abs(vbs["mean_regret"]) < 1e-12 and abs(vbs["mean_cost_ratio"] - 1.0) < 1
manifest = json.loads((BASE / "split_manifest.json").read_text())
assert not set(manifest["train_instance_ids"]) & set(manifest["test_instance_ids
print("integrity checks passed: labels, VBS invariants, disjoint split")
```

```
WARNING:root:Using pyllist instead of llist carries a performance penalty for vB
94-(S|P)I. Install the package with a command: $ pip install llist
WARNING: [cmt/CMT79-2P] can't use the external TSP solver (executable is not fou
nd?). Relying on internal TSP solver and the results may differ from those that
were published.
WARNING:root:Using pyllist instead of llist carries a performance penalty for MJ
76-INS. Install the package with a command: $ pip install llist
WARNING: [rfcs/Be83-RFCS] can't use the external TSP solver (executable is not f
ound?). Relying on internal TSP solver and the results may differ from those tha
t were published.
WARNING: [ty/Ty68-NN] can't use the external TSP solver (executable is not foun
d?). Relying on internal TSP solver and the results may differ from those that w
ere published.
integrity checks passed: labels, VBS invariants, disjoint split
```

13. Conclusions

1. **Per-instance algorithm selection works in-distribution**: the Random Forest meta-solver beats the single-best-solver baseline on routing regret ($\hat{m} = 0.741$, within-1% 63% vs 45%), consistent with the reference paper's algorithm-selection idea at student scale.
2. **The engineered features are useful predictive signals**: demand structure is prominent in impurity importance, held-out permutation importance highlights depot geometry and radial ring occupancy, and raw size is comparatively weak. These are associations, not causal claims.
3. **Transfer to real benchmarks is partial** ($\hat{m} \approx 0.98$ OOD) — measured, not assumed; better within-1% rates persist, while mean regret is close to SBS.

Loading [MathJax]/extensions/Safe.js

4. **The speed-quality trade-off is quantified**: HGS, used only as an external reference, buys ~3.5% better routes for ~91× the compute — fast selection and heavyweight search serve different regimes.

Limitations: synthetic training data; portfolio-relative labels (VBS ≈ +2% vs optima); runtime bounded but not rewarded; weak MTZ exact formulation under a free license; larger benchmark families (M, Golden, X) untested.

14. Reproduction

```
uv sync                                                  # environment
(Python >= 3.11)
uv run pytest                                            # test suite
uv run python -m cvrp_algorithm_selection.exact_baseline    # exact-vs-
heuristic + benchmark demo
uv run python -m cvrp_algorithm_selection.cvrplib_ood       # CVRPLIB
A/B/E/P evaluation
uv run python -m cvrp_algorithm_selection.tuning            # grid-
search supplement
uv run python -m cvrp_algorithm_selection.hgs_benchmark     # PyVRP HGS
extension (~15 min)
uv run python -m cvrp_algorithm_selection.ml_experiment     # FULL
frozen experiment (slow; overwrites nothing if artifacts exist)
```

The notebook itself loads frozen artifacts in fast mode. The commands above reproduce or refresh the corresponding experiment artifacts; some commands, especially HGS, are not fast.

Loading [MathJax]/extensions/Safe.js